Survival Estimation with Proportional Hazards Models

Version 1.2

Rune H B Christensen

30 November 2025

Contents

Introduction

Proportional hazard models are conventionally used to estimate hazard ratios, but they can also be a route to estimation of survival curves. This note is a primer on survival estimation with proportional hazard models using the **survival** package in **R**.

In the Cox proportional hazard model the baseline hazard is neither specified nor estimated resulting in a fully relative model fit. Estimated model coefficients describe how the hazard changes multiplicatively between covariate settings relative to a baseline hazard. The survival probabilities of a survival curve are absolute and therefore require the estimation of a baseline hazard. This note describes how this extra step is carried out and how the functions `basehaz`, `predict`, and `survfit` from the **survival** package can facilitate the estimation of hazards and survival curves.

A sometimes confusing aspect is that the implementation in the **survival** package is based on approximately centering the covariates. For example `help(predict.coxph)` says that "The `coxph` routines try to approximately center the predictors out of self protection." This is because there is a risk of numerical overflow when computing `exp(x)` when `x` is large. The functions `basehaz`, `predict`, and `survfit` all inherit the centered covariates and by default return results for the transformed covariates.

This note describes when and how the numerical challenges occur, how the Cox methods in the **survival** package deal with it and how the user will have to manoeuvre the options of the software to compute the quantities of interest.

We will use the following packages for examples in **R**:

```
library(data.table)
library(survival)
library(ggplot2)
library(ggfortify) # fortify.survfit
```

Illustrations in **R** use the **data.table** package for data handling and manipulation. Readers unfamiliar with the **data.table** package will hopefully (and probably) find the code easy to read.

The goal is to link theory to practice and show how certain survival analysis quantities are computed and how they are obtained using the **survival** package. The hope is that the reader will see a close resemblance between mathematical equations and the illustrated **R** code.

The first section outline the theory and notation for proportional hazards models and the associated survival curve estimation. After introduction of a very simple dataset with data on only 8 subjects and two covariates follows sections on (1) baseline hazard estimation using the `basehaz` function (2) predictions using `predict` and (3) survival curve estimation using `survfit`. In (1) we consider the topics of numerical overflow and choice of reference point for baseline hazard estimation. We end with a brief section on the estimation of marginal survival curves.

Preliminary theory

The Proportional hazards model is

$$h(t; x_i) = h_0(t) \exp(\eta_i), \text{ where } \eta = X\beta$$

Here β is a vector of model coefficients, X is a matrix of covariates, η is the linear predictor, $h_0(t)$ is the *baseline hazard* and $h(t, x_i)$ is the hazard function for the i'th covariate vector.

Often analyses stop here and inference is based on the hazard ratios provided by the estimated coefficients $\hat{\beta}$. If interest is in survival or risk curves, however, we need the following considerations.

The cumulative (or *integrated*) hazard is defined as

$$H(t) = \int_0^t h(u)\, \mathrm{d}u$$

which is directly related to the survival function by

$$S(t) = P(T > t) = \exp[-H(t)]$$

The proportional hazards model implies that the cumulative hazard may be written

$$H(t; x_i) = H_0(t) \exp(\eta_i)$$

where $H_0(t)$ is the *baseline cumulative hazard*. The associated survival function is

$$S(t; x_i) = \exp[-H_0(t) \exp(\eta_i)] = S_0(t)^{\exp(\eta_i)} \quad \text{where} \quad S_0(t) = \exp[-H_0(t)]$$

or equivalently

$$\log S(t; x_i) = \exp(\eta_i) \log S_0(t) = -H_0(t) \exp(\eta_i)$$

Based on an estimate of the baseline cumulative hazard it is therefore possible to compute the survival function for an individual with any covariate setting using the estimated parameters from the proportional hazards model:

$$\log \hat{S}(t; x_i) = -\hat{H}_0(t) \exp(x_i^\top \hat{\beta})$$

Fitting the Cox proportional hazards model provides the parameter estimates $\hat{\beta}$, but the baseline cumulative hazard function $H_0(t)$ requires an extra step.

An often cited estimator of the cumulative hazard is the Breslow estimator:

$$\hat{H}(t) = \sum_{i:t_i \le t} \frac{d_i}{\sum_{i:t_i \ge t} \exp(x_i^\top \hat{\beta})}$$

but there are other options when there are tied event times in the data.

Example data

Consider the following time-to-event data on 8 subjects with the covariates sex and age adopted from this blog post:

```
dt <- data.table(time = c(1,3,5,6,2,7,9,11),
                 status = c(1,0,1,1,1,0,1,1),
                 sex = c(1,1,1,1,0,0,0,0),
                 age = c(57, 52, 48, 42, 39, 31, 26, 22))
setorder(dt, time)
dt
```

```
##     time status   sex   age
##    <num>  <num> <num> <num>
## 1:     1      1     1    57
## 2:     2      1     0    39
## 3:     3      0     1    52
## 4:     5      1     1    48
## 5:     6      1     1    42
## 6:     7      0     0    31
## 7:     9      1     0    26
## 8:    11      1     0    22
```

A Cox proportional hazards model fit to these data provide the estimated model coefficients $\hat{\beta}$, but no estimate of the baseline hazard function:

```
fm <- coxph(Surv(time, status) ~ age + sex, data=dt)
summary(fm)
```

```
## Call:
## coxph(formula = Surv(time, status) ~ age + sex, data = dt)
##
##   n= 8, number of events= 6
##
##           coef  exp(coef)   se(coef)      z Pr(>|z|)
## age  0.6338168  1.8847908  0.3917450  1.618    0.106
## sex -7.4935992  0.0005566  5.1135367 -1.465    0.143
##
##     exp(coef) exp(-coef) lower .95 upper .95
## age 1.8847908     0.5306 8.746e-01     4.062
## sex 0.0005566  1796.5065 2.471e-08    12.538
##
## Concordance= 0.905  (se = 0.076 )
## Likelihood ratio test= 10.62  on 2 df,   p=0.005
## Wald test            = 2.72  on 2 df,   p=0.3
## Score (logrank) test = 7  on 2 df,   p=0.03
```

The baseline hazard

To estimate the baseline hazard function, we can conveniently use the `basehaz` function:

```
basehaz(fm)
```

```
##         hazard time
## 1 2.780808e-02    1
## 2 4.800556e-01    2
## 3 4.800556e-01    3
## 4 8.858100e+00    5
```

```
## 5 1.532633e+02    6
## 6 1.532633e+02    7
## 7 5.369320e+03    9
## 8 7.641099e+04   11
```

The `hazard` column in the returned `data.frame` is confusingly, however, not the baseline hazard function, $h_0(t)$ that one might expect for two main reasons:

1. What is denoted `hazard` is in fact a *cumulative* hazard function, $H(t)$
2. The provided cumulative hazard is not a *baseline* cumulative hazard in the sense that it is an estimate of $\hat{H}_0(t)$ in the equation $\log \hat{S}(t; x_i) = -\hat{H}_0(t) \exp(x_i^\top \hat{\beta})$.

What is, by default, returned by `basehaz` is instead the cumulative hazard for an approximately mean centered set of covariates: $H(t; \bar{x}) = H_0(t) \exp(\bar{x}^\top \hat{\beta})$, where $\bar{x}$ is a vector of approximately mean centered covariates.

The means of the covariates are stored in the model object so we can see what they are:

```
fm$means
```

```
##    age    sex
## 39.625  0.000
```

Two questions arise from this:

1. How can we obtain an estimate of $H_0(t)$, and
2. Why is the user interface so convoluted - it seems to almost resist the computation of $H_0(t)$

It appears that the answer to the second question has two parts: First, there is a risk that the exponential function will *overflow* leading to a loss of precision, and second, it appears that we don't actually need to compute $H_0(t)$.

The following two sections will explore these two reasons.

Now, however, we consider how to estimate $H_0(t)$; we simply specify `centered = FALSE` in `basehaz`:

```
basehaz(fm, centered = FALSE)
```

```
##         hazard time
## 1 3.442456e-13    1
## 2 5.942770e-12    2
## 3 5.942770e-12    3
## 4 1.096574e-10    5
## 5 1.897298e-09    6
## 6 1.897298e-09    7
## 7 6.646862e-08    9
## 8 9.459174e-07   11
```

Numerical overflow

Numerical overflow occurs because there is a limit to how large numbers it is possible to accurately compute the power of. For instance, on my computer (it may vary from one model to another), the largest whole number that I can take the exponential of without hitting infinity is 709:

```
c(exp(709), exp(710)) # 8.218407e+307          Inf
```

```
## [1] 8.218407e+307          Inf
```

The limit is stored in `.Machine$double.max.exp` and described as "the smallest positive power of `double.base` that overflows. Normally `1024`." (see `help(.Machine)`). Additionally `double.base` (see `.Machine$double.base`) is usually `2`. Therefore we should be able to compute 2^{1023} but not 2^{1024}:

```
c(2^1023, 2^1024) # 8.988466e+307           Inf
```

```
## [1] 8.988466e+307           Inf
```

The inaccuracies appear with much lower exponents because the computer can only hold around 16 digits in double precision. Therefore numbers greater than around 10^{16} will have zero correct decimals.

Because $2^x = 10^{\log_{10}(2^x)}$, we can compute the exponent to base 10, ie., the y such that $2^x = 10^y$ as $y = \log_{10}(2^x) = x \log_{10}(2)$. Equivalently we can compute the z such that $2^x = e^z$ by using the natural logarithm instead of the logarithm with base 10.

Now consider for example $x = 100$, then the exponents are:

```
x <- 100
y <- log10(2) * x
z <- log(2) * x
c(x=x, y=y, z=z)
```

```
##         x         y         z
## 100.00000  30.10300  69.31472
```

Because $2^{100} \approx 10^{30}$ the error in the computational representation of the numbers will be around 10^{15}. This shows up if we take the difference between the numbers 2^x, 10^y, and e^z. These numbers are mathematically identical, but because the computer only has around 16 digits to work with, the numbers are represented with error:

```
c(Diff_xy = 2^x - 10^y, Diff_xz = 2^x - exp(z))
```

```
##       Diff_xy       Diff_xz
##  3.659175e+15 -2.251800e+15
```

When interest is in computing a survival curve using $\log \hat{S}(t; x_i) = -\hat{H}_0(t) \exp(x_i^\top \hat{\beta})$ directly, it involves computing $\hat{H}_0(t)$ and $\exp(x_i^\top \hat{\beta})$ and multiplying them. If, however, one is large and the other is small, the large one will risk overflowing leading to a loss of precision. This situation happens when the values of x are far from zero. A much better situation is when extreme values of these two quantities are avoided. This can be achieved by centering the covariates appropriately as we shall explore in the next section.

The reference point is arbitrary

Let

$$H_0(t; x_i) = H_0(t) \exp(x_i^\top \beta)$$

denote the cumulative hazard function using zero as a reference point. Then choose $y_i = x_i - z$ for a new reference point z so we may write the cumulative hazard function as

$$H_z(t; x_i) = H_0(t; y_i) = H_0(t) \exp[(x_i - z)^\top \beta]$$

The relation between the cumulative hazards with different points of reference therefore amounts to a single multiplicative scaling factor:

$$H_0(t; x_i) = H_z(t; x_i) \exp(z^\top \beta)$$

For our model fit the scaling factor is

```
(scale_factor <- c(exp(fm$means %*% coef(fm))))
```

```
## [1] 80779771510
```

and the cumulative hazard using zero as reference point is:

```
cbind(basehaz(fm, centered = FALSE)$hazard,
      basehaz(fm, centered = TRUE)$hazard / scale_factor) |> head(3)
```

```
##              [,1]         [,2]
## [1,] 3.442456e-13 3.442456e-13
## [2,] 5.942770e-12 5.942770e-12
## [3,] 5.942770e-12 5.942770e-12
```

Importantly the change of reference point has no influence on the estimated parameters; mean-centering `age` gives the same coefficient, standard error and p-value for `age` in the Cox model:

```
dt[, c_age := age - mean(age)]
coxph(Surv(time, status) ~ c_age + sex, data=dt) |> summary() |>
  coef() |> round(digits=3)
```

```
##         coef exp(coef) se(coef)      z Pr(>|z|)
## c_age  0.634     1.885    0.392  1.618    0.106
## sex   -7.494     0.001    5.114 -1.465    0.143
```

Recall that $\exp(\beta_{age})$ is the multiplicative change in the hazard rate with a one-unit increase in `age`. That change is the same whether `age` is measured since birth or since some other reference z.

Nelson-Aalen and Breslow estimators

A simple estimator of the cumulative hazard is the Nelson-Aalen estimator using only information on the event indicator and the risk set. There is no consideration of covariates and it is therefore model free:

$$\hat{H}_{NA}(t) = \sum_{i:t_i \leq t} \frac{d_i}{n_i}$$

where d_i is one if t_i is the time of an event and zero otherwise, and n_i is number at risk.

We compute the Nelson-Aalen cumulative hazard estimate for our data (assuming it is ordered by increasing observation times) as follows:

```
dt[, n := rev(1:(.N))] # number at risk.
dt[, cum_haz := cumsum(status / n)]
dt[]
```

```
##     time status   sex   age   c_age     n   cum_haz
##    <num>  <num> <num> <num>   <num> <int>     <num>
## 1:     1      1     1    57  17.375     8 0.1250000
## 2:     2      1     0    39  -0.625     7 0.2678571
## 3:     3      0     1    52  12.375     6 0.2678571
## 4:     5      1     1    48   8.375     5 0.4678571
## 5:     6      1     1    42   2.375     4 0.7178571
## 6:     7      0     0    31  -8.625     3 0.7178571
## 7:     9      1     0    26 -13.625     2 1.2178571
## 8:    11      1     0    22 -17.625     1 2.2178571
```

When there are no covariates (and no ties among event times) in the Cox model, the cumulative hazard estimate simplifies to that of the Nelson-Aalen:

```
coxph(Surv(time, status) ~ 1, data=dt) |> basehaz()
```

```
##      hazard time
## 1 0.1250000    1
## 2 0.2678571    2
## 3 0.2678571    3
## 4 0.4678571    5
## 5 0.7178571    6
## 6 0.7178571    7
```

```
## 7 1.2178571    9
## 8 2.2178571   11
```

When there are covariates in the model, the cumulative hazard estimate is that of the Breslow estimator. We can compute the Breslow estimate as follows:

```
b <- coef(fm)
X <- model.matrix(~ 0 + age + sex, data=dt)
dt$eta <- c(X %*% b)
dt[, risk := exp(eta)]

dt$breslow <- sapply(dt$time, function(t) {
  dt[time == t, status] / dt[time >= t, sum(risk)]
}) |> cumsum()
print(dt, digits=3)
```

```
## Index: <time>
##     time status   sex   age   c_age     n cum_haz   eta     risk  breslow
##    <num>  <num> <num> <num>   <num> <int>   <num> <num>    <num>    <num>
## 1:     1      1     1    57  17.375     8   0.125  28.6 2.73e+12 3.44e-13
## 2:     2      1     0    39  -0.625     7   0.268  24.7 5.44e+10 5.94e-12
## 3:     3      0     1    52  12.375     6   0.268  25.5 1.15e+11 5.94e-12
## 4:     5      1     1    48   8.375     5   0.468  22.9 9.08e+09 1.10e-10
## 5:     6      1     1    42   2.375     4   0.718  19.1 2.03e+08 1.90e-09
## 6:     7      0     0    31  -8.625     3   0.718  19.6 3.41e+08 1.90e-09
## 7:     9      1     0    26 -13.625     2   1.218  16.5 1.43e+07 6.65e-08
## 8:    11      1     0    22 -17.625     1   2.218  13.9 1.14e+06 9.46e-07
```

A more direct but less readable implementation is:

```
dt[, denom := rev(cumsum(rev(risk)))]
dt[, breslow := cumsum(status / denom)]
print(dt[, -c("sex", "age", "c_age")], digits=3)
```

```
##     time status     n cum_haz   eta     risk  breslow    denom
##    <num>  <num> <int>   <num> <num>    <num>    <num>    <num>
## 1:     1      1     8   0.125  28.6 2.73e+12 3.44e-13 2.90e+12
## 2:     2      1     7   0.268  24.7 5.44e+10 5.94e-12 1.79e+11
## 3:     3      0     6   0.268  25.5 1.15e+11 5.94e-12 1.24e+11
## 4:     5      1     5   0.468  22.9 9.08e+09 1.10e-10 9.64e+09
## 5:     6      1     4   0.718  19.1 2.03e+08 1.90e-09 5.59e+08
## 6:     7      0     3   0.718  19.6 3.41e+08 1.90e-09 3.57e+08
## 7:     9      1     2   1.218  16.5 1.43e+07 6.65e-08 1.55e+07
## 8:    11      1     1   2.218  13.9 1.14e+06 9.46e-07 1.14e+06
```

All agree with the results of `basehaz`:

```
all.equal(basehaz(fm, centered=FALSE)$hazard, dt$breslow)
```

```
## [1] TRUE
```

Bear in mind that the computations above only make sense when there are no ties among the event times. If there are tied event times, there are several methods to handle that ("efron", "breslow", "exact"; see the `ties` argument of `coxph`). We will not consider ties any further in this note.

Predictions from the Cox model

The predict method for Cox models (model objects of class `coxph`) has several options of which three are of primary importance:

1. `newdata`,
2. `type=c("lp", "risk", "expected", "terms", "survival")`, and
3. `reference=c("strata", "sample", "zero")`

If `newdata` is provided (a `data.frame` with specified value of the covariates), predictions are provided for those predictor values. If `newdata` is not provided predicted values are provided for the covariate values in the data used to fit the model. In the following we consider the situation in which `newdata` is not provided but give an example of its use for `type = "survival"`.

As was the case with `basehaz`, the `predict` method will provide predictions relative to the *mean centered* covariates by default, that is, when `reference = "strata"`. In fact, it will also do that when `reference = "sample"` the difference being whether mean centering is done within strata or on the whole sample. The difference is only relevant when the Cox model contains a specification of strata; otherwise they are the same. To obtain predictions for the original covariates we can specify `reference = "zero"`.

A summary of the different quantities computed for the different values of the `type` argument are provided in the following list:

- lp: linear predictor $\hat{\eta} = X\hat{\beta}$
- risk: exponentiated linear predictor $\exp(\hat{\eta}) = \exp(X\hat{\beta})$
- expected: cumulative hazard function $H(t_i; x_i) = H_0(t_i)\exp(x_i^\top \hat{\beta})$
- survival: survival probabilities $S(t_i; x_i) = \exp[-H_0(t_i)\exp(x_i^\top \hat{\beta})]$
- terms: a matrix with the contributions to the linear predictor from each model term in separate columns

type = "lp"

The `type` argument controls what is predicted. Here `type = "lp"` (the default) provides the linear predictor, η.

Here we compute the linear predictor, $\hat{\eta} = X\hat{\beta}$ by hand and using the predict method:

```
X <- model.matrix(~ 0 + age + sex, data=dt)
lp <- c(X %*% coef(fm))
cbind(lp,
      predict(fm, type = "lp", reference = "zero")) |> head(3)
```

```
##         lp
## 1 28.63396 28.63396
## 2 24.71886 24.71886
## 3 25.46488 25.46488
```

The model object also contains a `linear.predictors` object, which is the mean centered linear predictor:

```
cbind(fm$linear.predictors,
      predict(fm, type = "lp")) |> head(3)
```

```
##            [,1]       [,2]
## [1,]  3.5189684  3.5189684
## [2,] -0.3961355 -0.3961355
## [3,]  0.3498842  0.3498842
```

The following are equivalent ways of obtaining the linear predictor for the mean centered covariates (results not shown):

```
predict(fm, type = "lp", reference = "sample") # same (no strata in model)
predict(fm, type = "lp", reference = "strata") # same (no strata in model)
predict(fm) # same (due to default arguments)
```

type = "risk"

Type "risk" provides the exponentiated linear predictor $\exp(\hat{\eta}) = \exp(X\hat{\beta})$:

```
cbind(exp(lp),
      predict(fm, type = "risk", reference = "zero")) |> head(3)
```

```
##           [,1]         [,2]
## 1 2.726285e+12 2.726285e+12
## 2 5.435796e+10 5.435796e+10
## 3 1.146187e+11 1.146187e+11
```

type = "expected"

Type "expected" provides an estimate of the cumulative hazard function $H(t_i; x_i) = H_0(t_i)\exp(x_i^\top \hat{\beta})$ for each row (i) of the dataset:

```
H0 <- basehaz(fm, centered = FALSE)$hazard
cbind(H0 * exp(lp),
      predict(fm, type = "expected", reference = "zero"),
      predict(fm, type = "expected", reference = "sample")) |> head(3)
```

```
##        [,1]      [,2]      [,3]
## 1 0.9385114 0.9385114 0.9385114
## 2 0.3230369 0.3230369 0.3230369
## 3 0.6811525 0.6811525 0.6811525
```

Because this is an absolute measure rather than a relative measure the reference argument is ineffective.

When computing these quantities by hand we may want to avoid the `H0 * exp(lp)` construction: `H0` contain very small numbers (around 10^{-10}) and `exp(lp)` contain rather large numbers (around 10^{10}): A recipe for numerical problems and loss of precision. As explained in the previous section on numerical overflow a better strategy is to chose a reference closer to the mean of the linear predictor rather than zero as illustrated by the following code:

```
basehaz(fm, centered=TRUE)$hazard * predict(fm, type = "risk", reference = "sample")
```

```
## [1] 0.9385114 0.3230369 0.6811525 0.9959573 0.3843788 0.6475801 0.9538031
## [8] 1.0755799
```

Now both terms of the product are using the sample means as a point of reference instead of zero. The computed quantity (the cumulative hazard) is exactly the same but the computation is much safer because both terms of the product are closer to unity.

Note that $H(t_i; x_i)$ is the cumulative hazard for the covariates x_i evaluated at the time t_i.

type = "survival"

Type "survival" computes the quantities $S(t_i; x_i) = \exp[-H_0(t_i)\exp(x_i^\top \hat{\beta})]$

```
cbind(exp(-H0 * exp(lp)),
      predict(fm, type = "survival", reference = "sample"),
      predict(fm, type = "survival", reference = "zero")) |> head(3)
```

```
##        [,1]      [,2]      [,3]
## 1 0.3912098 0.3912098 0.3912098
## 2 0.7239472 0.7239472 0.7239472
## 3 0.5060335 0.5060335 0.5060335
```

and again the quantities are absolute rendering the `reference` argument ineffective.

As was the case with the cumulative hazard, $S(t_i; x_i)$ is a quantity (here a survival probability) for the covariates x_i evaluated at the time t_i. The vector of numbers produced by `predict` with `type = "survival"` therefore do not constitute a survival curve in itself. Rather the numbers are points on survival curves for different covariate settings evaluated at different time points.

The `newdata` argument can be used to specify the covariate settings and time points of interest. If, for example, we are interested in the survival curve for the fourth subject, that is the survival probability at all observed event or censoring times, we can use:

```
ndata <- dt[, .(time, status, age=age[4], sex=sex[4])]
ndata$pred <- predict(fm, ndata, type="survival")
ndata
```

```
##     time status   age   sex          pred
##    <num>  <num> <num> <num>         <num>
## 1:     1      1    48     1  9.968783e-01
## 2:     2      1    48     1  9.474559e-01
## 3:     3      0    48     1  9.474559e-01
## 4:     5      1    48     1  3.693697e-01
## 5:     6      1    48     1  3.282393e-08
## 6:     7      0    48     1  3.282393e-08
## 7:     9      1    48     1 6.568761e-263
## 8:    11      1    48     1  0.000000e+00
```

A more direct route to survival curve computation is provided by the `survfit` function discussed below.

type = "terms"

A "terms" prediction is a matrix with the contributions to the linear predictor from each model term in separate columns:

```
(XX <- predict(fm, type="terms"))
```

```
##           age       sex
## 1  11.0125677 -7.493599
## 2  -0.3961355  0.000000
## 3   7.8434834 -7.493599
## 4   5.3082161 -7.493599
## 5   1.5053150 -7.493599
## 6  -5.4666703  0.000000
## 7  -8.6357545  0.000000
## 8 -11.1710219  0.000000
## attr(,"constant")
## [1] 25.11499
```

The attribute `"constant"` is the centering value for the linear predictor obtained as the product of the covariate means and the model coefficients:

```
c(fm$means %*% coef(fm))
```

```
## [1] 25.11499
```

The row sums of the term predictions equal the linear predictor:

```
cbind(rowSums(XX),
      predict(fm, type = "lp", reference = "sample")) |> head(3)
```

```
##         [,1]       [,2]
## 1  3.5189684  3.5189684
## 2 -0.3961355 -0.3961355
## 3  0.3498842  0.3498842
```

and if we add the attribute, "constant", then we obtain the zero-reference linear predictor:

```
cbind(rowSums(XX) + attr(XX, "constant"),
      predict(fm, type = "lp", reference = "zero")) |> head(3)
```

```
##       [,1]     [,2]
## 1 28.63396 28.63396
## 2 24.71886 24.71886
## 3 25.46488 25.46488
```

Survival curve estimation using survfit

Calling `survfit` on our Cox model fit provides rather limited information:

```
(sfm <- survfit(fm))
```

```
## Call: survfit(formula = fm)
##
##      n events median 0.95LCL 0.95UCL
## [1,] 8      6      5       2      NA
```

The resulting object, however, contains more information:

```
str(sfm)
```

```
## List of 15
##  $ n        : int 8
##  $ time     : num [1:8] 1 2 3 5 6 7 9 11
##  $ n.risk   : num [1:8] 8 7 6 5 4 3 2 1
##  $ n.event  : num [1:8] 1 1 0 1 1 0 1 1
##  $ n.censor : num [1:8] 0 0 1 0 0 1 0 0
##  $ surv     : num [1:8] 9.73e-01 6.19e-01 6.19e-01 1.42e-04 2.75e-67 ...
##  $ cumhaz   : num [1:8] 0.0278 0.4801 0.4801 8.8581 153.2633 ...
##  $ std.err  : num [1:8] 0.0684 0.6642 0.6642 20.7056 568.0057 ...
##  $ logse    : logi TRUE
##  $ std.chaz : num [1:8] 0.0684 0.6642 0.6642 20.7056 568.0057 ...
##  $ lower    : num [1:8] 8.51e-01 1.68e-01 1.68e-01 3.38e-22 0.00 ...
##  $ upper    : num [1:8] 1 1 1 1 1 1 NA NA
##  $ conf.type: chr "log"
##  $ conf.int : num 0.95
##  $ call     : language survfit(formula = fm)
##  - attr(*, "class")= chr [1:2] "survfitcox" "survfit"
```

To prepare for plotting the survival curve, we can extract some of the relevant data:

```
plotdata <- with(sfm, data.table(time, n.risk, n.event, cumhaz, surv))
print(plotdata, digits=3)
```

```
##     time n.risk n.event   cumhaz     surv
##    <num>  <num>   <num>    <num>    <num>
```

```
## 1:     1      8       1 2.78e-02 9.73e-01
## 2:     2      7       1 4.80e-01 6.19e-01
## 3:     3      6       0 4.80e-01 6.19e-01
## 4:     5      5       1 8.86e+00 1.42e-04
## 5:     6      4       1 1.53e+02 2.75e-67
## 6:     7      3       0 1.53e+02 2.75e-67
## 7:     9      2       1 5.37e+03 0.00e+00
## 8:    11      1       1 7.64e+04 0.00e+00
```

Here `cumhaz` is the cumulative hazard for the mean centered covariates $H_z(t;x)$ and `surv` is $\exp[-H_z(t;x)]$. We have previously used `basehaz` to compute the cumulative hazard, and indeed the same numbers are produced by this function:

```
cumhaz <- basehaz(fm, centered = TRUE)$hazard
cbind(cumhaz, surv=exp(-cumhaz)) |> signif(3)
```

```
##        cumhaz     surv
## [1,] 2.78e-02 9.73e-01
## [2,] 4.80e-01 6.19e-01
## [3,] 4.80e-01 6.19e-01
## [4,] 8.86e+00 1.42e-04
## [5,] 1.53e+02 2.75e-67
## [6,] 1.53e+02 2.75e-67
## [7,] 5.37e+03 0.00e+00
## [8,] 7.64e+04 0.00e+00
```

In fact, `basehaz` is an alias for the `survfit` function and calls that function internally with minimal post processing.

The information in `plotdata` can also be extracted with the following functions (results not shown):

```
summary(sfm)
fortify(sfm) ## packages ggplot2 and ggfortify
```

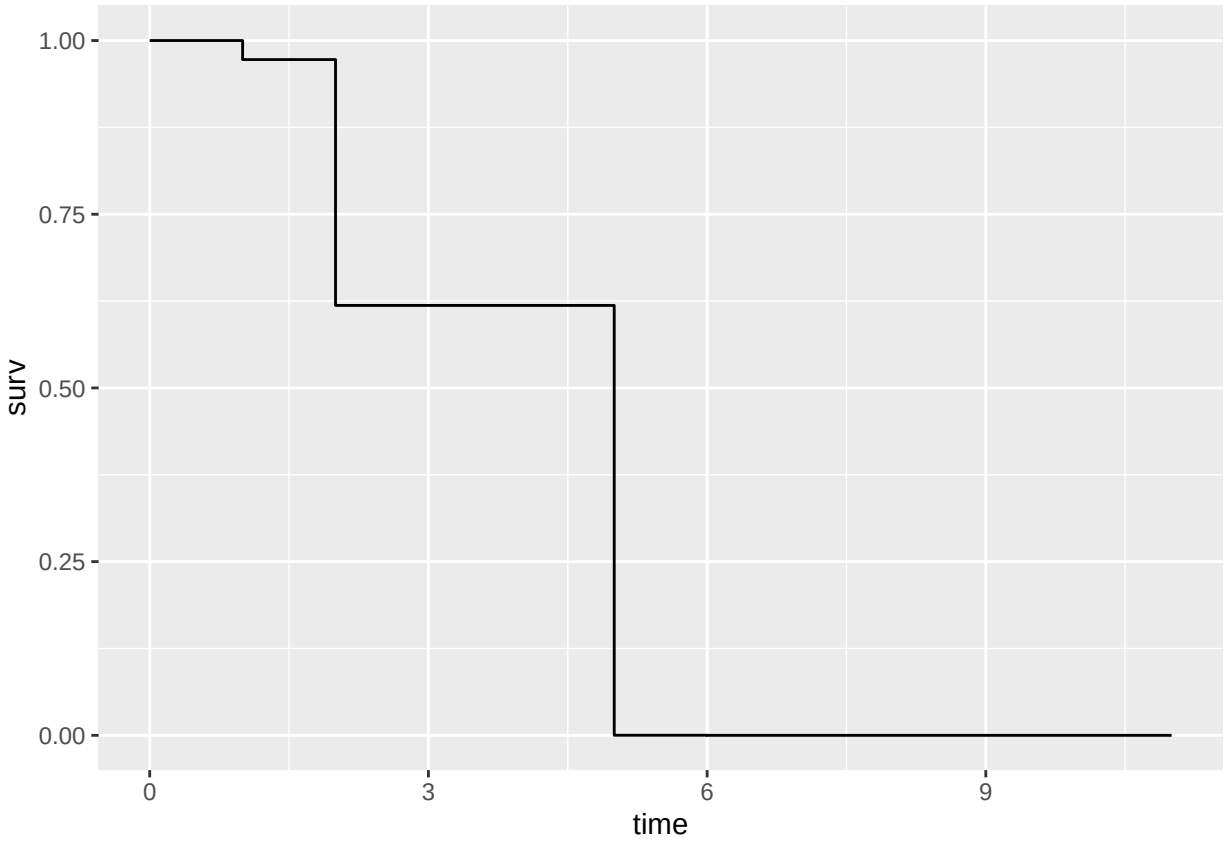
Another useful function is `survfit0`. This function adds a row to the data for time zero which is useful for plotting:

```
plotdata <- survfit0(sfm) |> fortify()
print(plotdata, digits=3)
```

```
##   time n.risk n.event n.censor     surv  std.err upper    lower
## 1    0      8       0        0 1.00e+00 0.00e+00     1 1.00e+00
## 2    1      8       1        0 9.73e-01 6.84e-02     1 8.51e-01
## 3    2      7       1        0 6.19e-01 6.64e-01     1 1.68e-01
## 4    3      6       0        1 6.19e-01 6.64e-01     1 1.68e-01
## 5    5      5       1        0 1.42e-04 2.07e+01     1 3.38e-22
## 6    6      4       1        0 2.75e-67 5.68e+02     1 0.00e+00
## 7    7      3       0        1 2.75e-67 5.68e+02     1 0.00e+00
## 8    9      2       1        0 0.00e+00 2.94e+04    NA       NA
## 9   11      1       1        0 0.00e+00 5.24e+05    NA       NA
```

A simple plot of the survival function is

```
ggplot(plotdata, aes(time, surv)) + geom_step()
```



The survival probabilities on this curve refer to a subject with values of the covariates equal to the approximately mean centered covariates, ie., with the following values of `age` and `sex`:

```
fm$means
```

```
##    age    sex
## 39.625  0.000
```

Whether this is a relevant subject to produce a survival curve for is context dependent. There is a risk, however, that the mean-centered values do not refer to any subject in the population, that the mean-center subject does not make sense (eg., 50% male and 50% female), or that the mean-center subject is unrepresentative of the population of interest.

Using the newdata argument

The `newdata` argument can be used to get survival curves for particular values of the covariates. If we want to see the survival curves for each sex at an average age value, we could specify the `newdata` argument as follows:

```
ndata <- expand.grid(age = dt[, mean(age)],
                     sex = dt[, unique(sex)])
ndata
```

```
##      age sex
## 1 39.625   1
## 2 39.625   0
```

The survival probabilities are obtained as before, but since there are two rows in `ndata` two survival curves

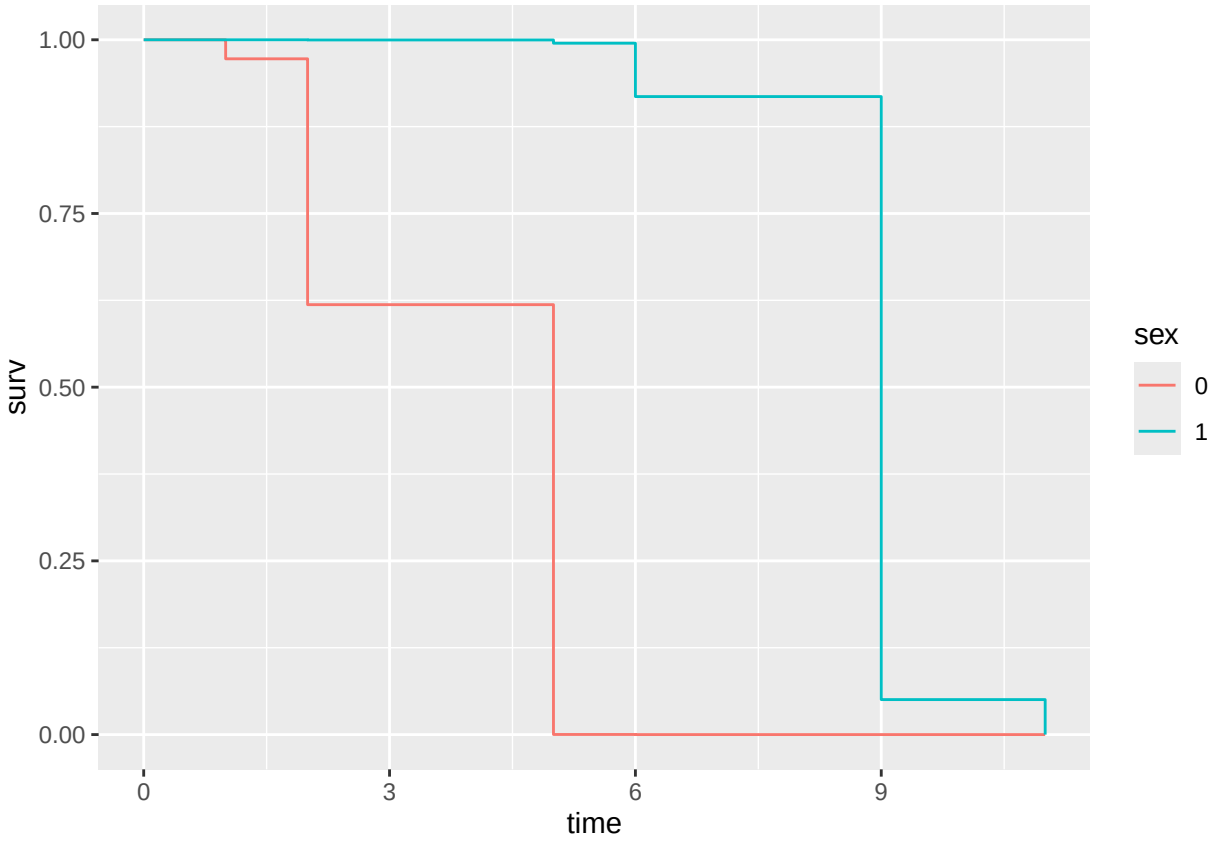
are produced. They are returned as a two-column matrix with one row for each unique time point:

```
sfm2 <- survfit(fm, newdata = ndata) |> survfit0()
str(sfm2, list.len = 7)
```

```
## List of 16
##  $ n        : int 8
##  $ time     : num [1:9] 0 1 2 3 5 6 7 9 11
##  $ n.risk   : num [1:9] 8 8 7 6 5 4 3 2 1
##  $ n.event  : num [1:9] 0 1 1 0 1 1 0 1 1
##  $ n.censor : num [1:9] 0 0 0 1 0 0 1 0 0
##  $ surv     : num [1:9, 1:2] 1 1 1 1 0.995 ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : NULL
##   .. ..$ : chr [1:2] "1" "2"
##  $ cumhaz   : num [1:9, 1:2] 0.00 1.55e-05 2.67e-04 2.67e-04 4.93e-03 ...
##   [list output truncated]
##  - attr(*, "class")= chr [1:3] "survfit0" "survfitcox" "survfit"
```

We extract, manipulate and plot the survival probabilities as follows:

```
dsurv <- with(sfm2, cbind(data.table(n, time, n.risk, n.event),
                          as.data.table(exp(-cumhaz))))
dsurv <- melt(dsurv, measure.vars = paste0("V", 1:2),
              variable.name = "row", value.name = "surv")
dsurv[, sex := factor(fifelse(row == "V1", 1, 0))]
ggplot(dsurv, aes(time, surv, color=sex)) + geom_step()
```



Note that while `sfm2$surv` should have contained the survival probabilities, a small bug requires us to use

`exp(-sfm2$cumhaz)` to obtain the survival probabilities as is further explained in the appendix.

To illustrate the variation in survival curves in the population, we might want to see the survival curves for each sex and for age values at the 25'th, 50'th and 75'th percentiles of the population values. To do this, we construct the prediction dataset (`newdata`) as follows:

```
ndata <- expand.grid(age = dt[, quantile(age, c(0.25, .5, .75))], # dt[, mean(age)],
                     sex = dt[, unique(sex)]) |> as.data.table()
ndata
```

```
##      age   sex
##    <num> <num>
## 1: 29.75     1
## 2: 40.50     1
## 3: 49.00     1
## 4: 29.75     0
## 5: 40.50     0
## 6: 49.00     0
```

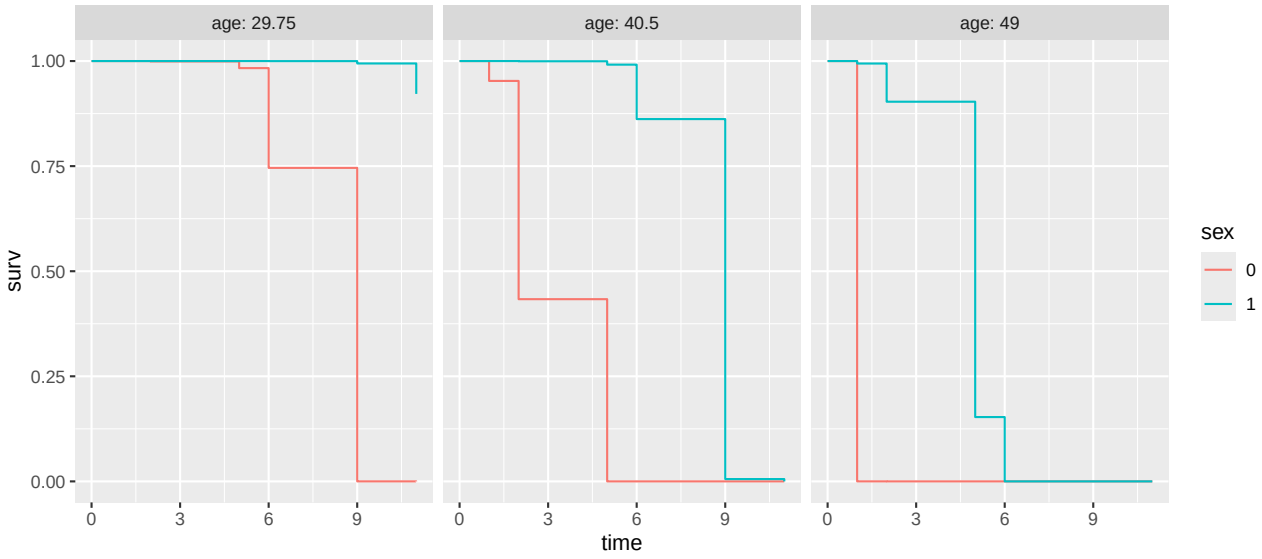
We obtain the six survival curves, flip the survival curve data to long format and combine with `ndata` as follows:

```
sfm3 <- survfit(fm, newdata = ndata) |> survfit0()
dsurv <- with(sfm3, cbind(data.table(n, time, n.risk, n.event),
                          as.data.table(exp(-cumhaz))))
dsurv <- melt(dsurv, measure.vars = paste0("V", 1:ncol(sfm3$cumhaz)),
              variable.name = "row", value.name = "surv")
ndata[, row := factor(paste0("V", 1:(.N)))]
dsurv <- merge(dsurv, ndata, by="row", all.x=TRUE)
dsurv[, `:=`(age = factor(age), sex = factor(sex))]
print(dsurv, topn = 3)
```

```
## Key: <row>
##        row     n  time n.risk n.event      surv    age    sex
##     <fctr> <int> <num>  <num>   <num>     <num> <fctr> <fctr>
##  1:     V1     8     0      8       0 1.0000000  29.75      1
##  2:     V1     8     1      8       1 1.0000000  29.75      1
##  3:     V1     8     2      7       1 0.9999995  29.75      1
## ---
## 52:     V6     8     7      3       0 0.0000000     49      0
## 53:     V6     8     9      2       1 0.0000000     49      0
## 54:     V6     8    11      1       1 0.0000000     49      0
```

and we are now ready to plot the survival curves:

```
ggplot(dsurv, aes(time, surv, color=sex)) + geom_step() +
  facet_wrap(~age, labeller="label_both")
```



Marginal survival curves

The survival curves obtained in the last section were *conditional* curves; they corresponded to particular individuals with set values of the covariates sex and age. If, instead, we are interested in a survival curve representing a population-average, we need a *marginal* survival curve.

A standard Kaplan-Meier curve is an option, but that curve is unadjusted for the covariates. Another option is a survival curve for a set of covariates set to their average value similar to that produced by `survfit` when the `newdata` argument is not set. This method, known as the *average covariate method*, has serious disadvantages (cf. Nieto and Coresh, 1996) and is recommended against in numerous texts. A better option is to standardize the survival curves to the distribution of covariates in the population of interest. Usually the population of interest is taken to be the study population and we will illustrate that in the following.

First, we extract the survival curves for each subject in the study population (8 subjects in our case) and then average the survival probability at each time point over the population. We do that by taking the row-means of the matrix of survival curves, `surv`.

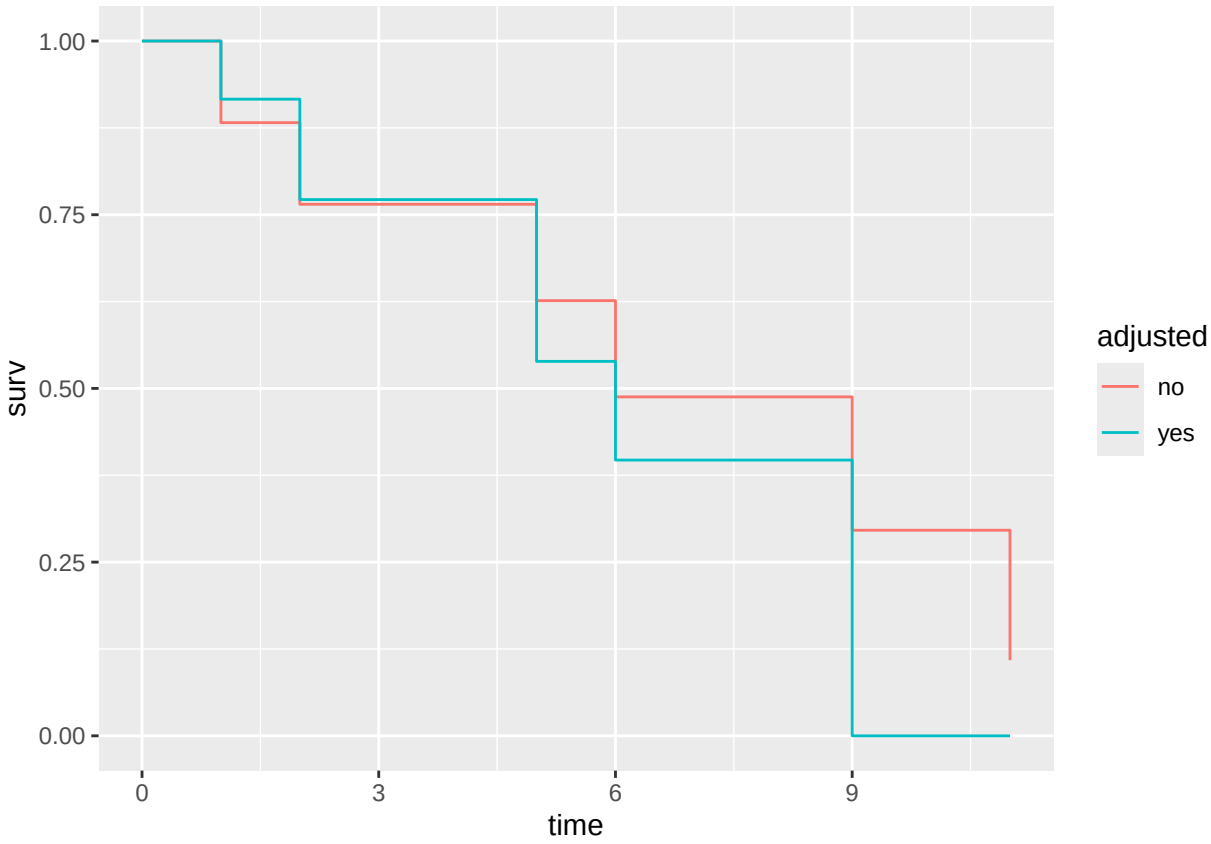
```
sfm4 <- survfit(fm, newdata = dt) |> survfit0()
dsurv <- with(sfm4, cbind(data.table(
  n, time, n.risk, n.event, surv = rowMeans(surv), adjusted="yes")))
```

For comparison we compute the unadjusted marginal survival curve. We do that using the Nelson-Aalen estimator of the cumulative hazard function which is what we would also get if we extracted the survival curve from a Cox model with no covariates:

```
dsurv2 <- copy(dsurv)[, `:=`(surv = exp(-cumsum(n.event / n.risk)),
                             adjusted = "no")]
```

The last step is to merge the data and plot the survival curves:

```
dsurv3 <- rbind(dsurv, dsurv2)
ggplot(dsurv3, aes(time, surv, color=adjusted)) + geom_step()
```



Appendix

Buglet in computation of survival probabilities in `survfit`

In the current version of the **survival** package (version 3.8.3) there is a small bug in the computation of survival probabilities, and I have reported it to the package author here. The issue involves only the computation of the survival probabilities, not the computation of the cumulative hazard (it appears that the survival computation is insufficiently guarded against underflow.) In the main text above we therefore computed the survival by exponentiating the negative of the cumulative hazard.

The following code shows the difference between the two objects: the built-in `surv` component of the `survfit` object and the exponential of the negative of the cumulative hazard stored as the `cumhaz` component of the `survfit` object:

```
ndata <- expand.grid(age = dt[, mean(age)],
                     sex = dt[, unique(sex)])
sfm2 <- survfit(fm, newdata = ndata) |> survfit0()
(s1 <- sfm2$surv)
```

```
##               1           2
##  [1,] 1.0000000 1.000000e+00
##  [2,] 0.9999845 9.725750e-01
##  [3,] 0.9997328 6.187490e-01
##  [4,] 0.9997328 6.187490e-01
##  [5,] 0.9950814 1.422251e-04
##  [6,] 0.9182259 2.745250e-67
##  [7,] 0.9182259 2.745250e-67
##  [8,] 0.0000000 0.000000e+00
##  [9,] 0.0000000 0.000000e+00
```

```
(s2 <- exp(-sfm2$cumhaz))
```

```
##               [,1]         [,2]
##  [1,] 1.000000e+00 1.000000e+00
##  [2,] 9.999845e-01 9.725750e-01
##  [3,] 9.997328e-01 6.187490e-01
##  [4,] 9.997328e-01 6.187490e-01
##  [5,] 9.950814e-01 1.422251e-04
##  [6,] 9.182259e-01 2.745250e-67
##  [7,] 9.182259e-01 2.745250e-67
##  [8,] 5.035003e-02 0.000000e+00
##  [9,] 3.373727e-19 0.000000e+00
```

The two objects appear identical until except for the last two rows, and this is confirmed by the following code:

```
all.equal(s1, s2, check.attributes = FALSE)
```

```
## [1] "Mean absolute difference: 0.02517501"
```

```
all.equal(s1[1:7, ], s2[1:7, ], check.attributes = FALSE)
```

```
## [1] TRUE
```

R session info:

```
sessionInfo()
```

```
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sequoia 15.6.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Copenhagen
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] ggfortify_0.4.19  ggplot2_3.5.1     survival_3.8-3    data.table_1.16.4
##
## loaded via a namespace (and not attached):
##  [1] Matrix_1.7-1      gtable_0.3.6      dplyr_1.1.4       compiler_4.4.2
##  [5] tinytex_0.54      tidyselect_1.2.1  stringr_1.5.1     gridExtra_2.3
##  [9] tidyr_1.3.1       splines_4.4.2     scales_1.3.0      yaml_2.3.10
## [13] fastmap_1.2.0     lattice_0.22-6    R6_2.5.1          labeling_0.4.3
## [17] generics_0.1.3    knitr_1.49        tibble_3.2.1      munsell_0.5.1
## [21] rprojroot_2.0.4   pillar_1.9.0      rlang_1.1.6       utf8_1.2.4
## [25] stringi_1.8.4     xfun_0.52         cli_3.6.3         withr_3.0.2
```

```
## [29] magrittr_2.0.3    digest_0.6.37     grid_4.4.2        rstudioapi_0.17.1
## [33] lifecycle_1.0.4   vctrs_0.6.5       evaluate_1.0.1    glue_1.8.0
## [37] farver_2.1.2      fansi_1.0.6       colorspace_2.1-1  rmarkdown_2.29
## [41] purrr_1.0.2       tools_4.4.2       pkgconfig_2.0.3   htmltools_0.5.8.1
```